Question: Implement a 4 bit ALU that takes two 4 bit numbers A(A3,A2,A1,A0) and B(B3,B2,B1,B0) and has S2, S1 & S0 as three selection switches to implement the given truth table.

S2	S1	S0	Output	Function
0	0	0	Ai	Buffer A
0	0	1	Ai'	NOT A
0	1	0	Bi	Buffer B
0	1	1	Bi'	NOT B
1	0	0	Ai AND Bi	AND
1	0	1	Ai OR Bi	OR
1	1	0	Ai XOR Bi	XOR

Design Instructions:

● Core Unit: Build a 4-bit ALU by cascading four 1-bit Full Adder slices.
● The Rule: You cannot use a ready-made MUX for input selection.

● The Steps:
1. Find what Xi and Yi must be for each selection line S2, S1, S0 combination.
2. Use K-Maps to find the simplest Boolean equations for Xi and Yi and Cin.
3. Build this selection logic using basic gates and connect it to the Full Adder.

Table-1: Condensed Function Table of the ALU Logic Unit

S0	S1	S2	Xi	Yi	Zi	Output
0	0	0	Ai	0	0	Ai
0	0	1	Ai	1	0	Ai’
0	1	0	0	Bi	0	Bi
0	1	1	1	Bi	0	Bi’
1	0	0	Ai.Bi	0	0	Ai AND Bi
1	0	1	Ai+Bi	0	0	Ai OR Bi
1	1	0	Ai	Bi	0	Ai XOR Bi

Explanation:

1.When Selection line- S0 S1 S2 = 000

Here the output is Ai, so A is passed directly. That’s why Xi = Ai is selected. Since no modification is needed, Yi = 0 and Zi = 0. This combination simply keeps A unchanged, so the output becomes Ai (Buffer A).

2.When Selection line- S0 S1 S2 = 001

Here the output is Ai′, meaning A must be inverted. So we still keep Xi = Ai, but now we need inversion, so Yi = 1. Zi = 0 because no extra operation is required. This produces NOT A.

3.When Selection line- S0 S1 S2 = 010
Here the output is Bi, so B is passed directly. That's why Yi = Bi is chosen as the main input, while Xi = 0 (A is not used). Zi = 0 since no operation is needed. This gives Buffer B.

4.When Selection line- S0 S1 S2 = 011
Here the output is Bi′, so B must be inverted. We keep Yi = Bi, but now inversion is needed, so Xi = 1 is used to flip it. Zi = 0. This produces NOT B.

5.When Selection line- S0 S1 S2 = 100
Here the output is Ai AND Bi. So both inputs are needed together. That's why Xi = Ai·Bi (AND combination is prepared directly), while Yi = 0 and Zi = 0. This gives the AND result.

6.When Selection line- S0 S1 S2 = 101
Here the output is Ai OR Bi. Again both inputs are needed, but now in OR form. So Xi = Ai + Bi is selected, Yi = 0, and Zi = 0. This produces the OR output.

7.When Selection line- S0 S1 S2 = 110
Here the output is Ai XOR Bi. So both inputs are used separately: Xi = Ai and Yi = Bi. With Zi = 0, the circuit produces XOR behavior. This gives the final XOR result.

Table-2: Truth Table of the ALU Logic Unit

S0	S1	S2	Bi	Ai	Xi	Yi	Zi	Output
0	0	0	0	0	0	0	0	BUFFER (Ai)
				1	1			
			1	0	0	0		
				1	1			
0	0	1	0	0	0	1	0	NOT (Ai)
				1	1			
			1	0	0	1		
				1	1			
0	1	0	0	0	0	0	0	BUFFER (Bi)
				1	0			
			1	0	0	1		
				1	0			
0	1	1	0	0	1	0	0	NOT (Bi)
				1	1			
			1	0	1	1		
				1	1			
1	0	0	0	0	0	0	0	AND (Ai AND Bi)
				1	0			
			1	0	0	0		
				1	1			
1	0	1	0	0	0	0	0	OR (Ai OR Bi)
				1	1			
			1	0	1	0		
				1	1			
1	1	0	0	0	0	0	0	XOR (Ai XOR Bi)
				1	1			
			1	0	0	1		
				1	1			

K-Map

1.K-map For Xi:

A=0

BC \ DE	00	01	11	10
00	0 (0)	0 (1)	1 (3)	1 (2)
01	0 (4)	0 (5)	1 (7)	1 (6)
11	1 (12)	1 (13)	1 (15)	1 (14)
10	0 (8)	0 (9)	0 (11)	0 (10)

A=1

BC \ DE	00	01	11	10
00	0 (16)	0 (17)	1 (19)	0 (18)
01	0 (20)	1 (21)	1 (23)	1 (22)
11	0 (28)	0 (29)	0 (31)	0 (30)
10	0 (24)	0 (25)	1 (27)	1 (26)

Simplified Boolean Expression

A'B'D + A'BC + AB'CE + ABC'D + B'CD

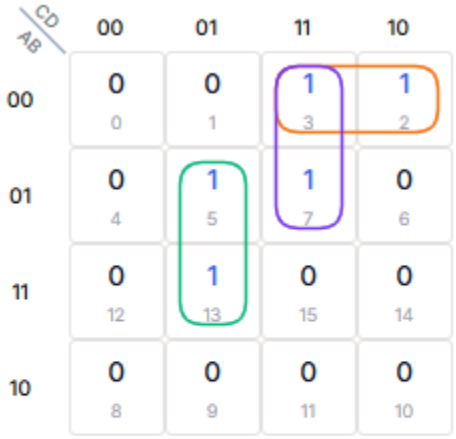
Let, A=S0, B=S1, C=S2, D=Ai, E=Bi

So,

The Final simplified boolean expression is:

X=S0’S1’Ai + S0’S1S2 + S0S1’S2Bi + S0S1S2’Ai + S1’S2Ai

2.K-Map For Yi:



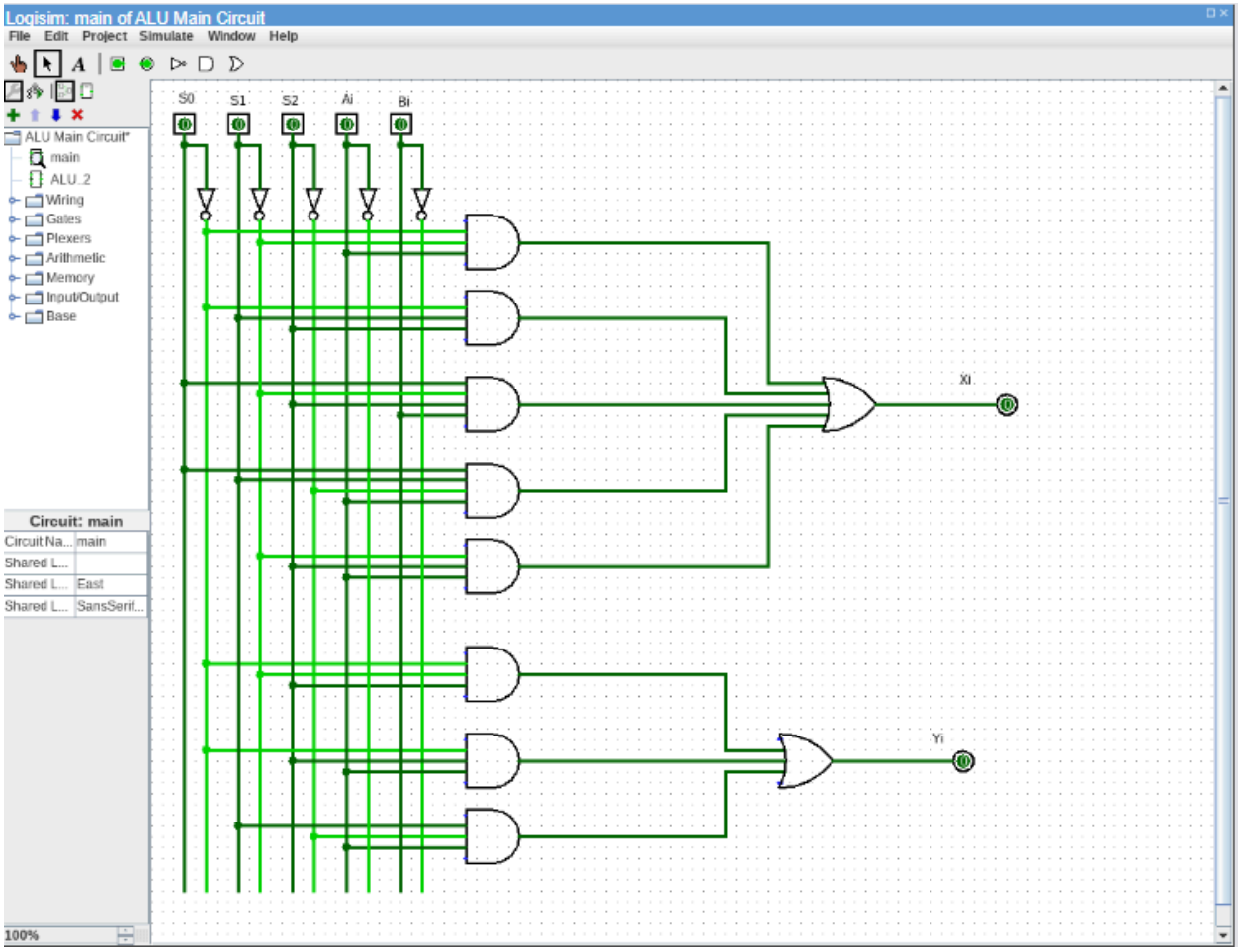
Simplified Boolean Expression

A'B'C + A'CD + BC'D

Again
Let, A=S0, B=S1, C=S2, D=Ai, E=Bi
So,
The Final simplified boolean expression is:

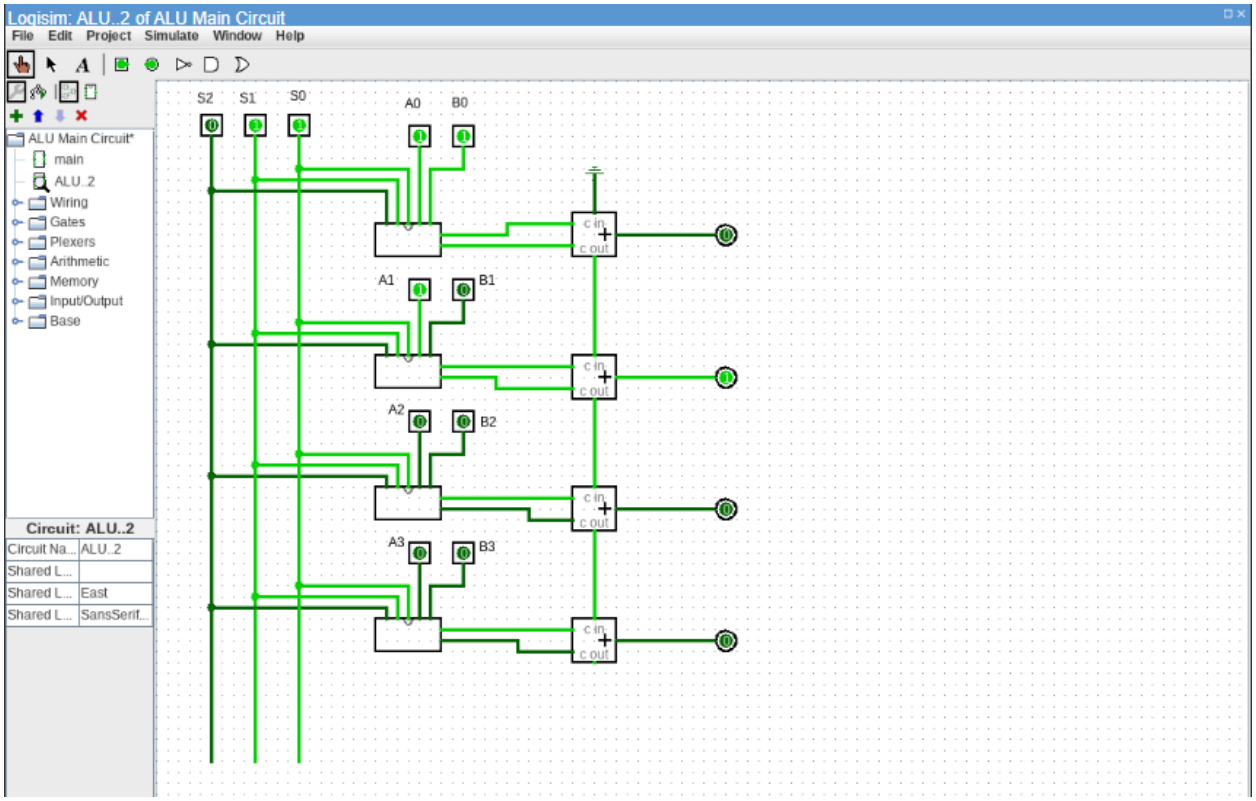
Y= S0’S1’S2 + S0’S2Ai + S1S2’Ai

ALU-Circuit



There is no gate for Zi here because we always get Zi=0. And don’t get Zi=1.

1-bit ALU



Logisim circuit attached in another file.